

HISTOGRAM EQUALIZATION IN OPENCL AND NUMPY

Case Study in Parallelizing Image Processing

H.Kirollos

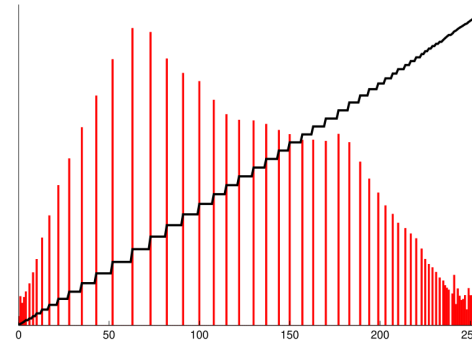
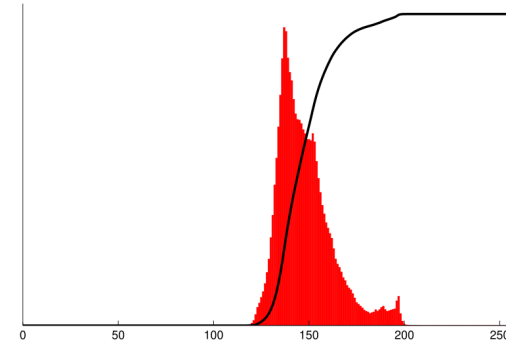
<https://github.com/alonzo-bazaar/parallel-programming>

CONTENTS

1. Histogram Equalization
2. Our Procedure, and Parallel Patterns Therein
3. Technical Choices
4. OpenCL Implementation: outline
5. NumPy Implementation
6. Benchmarks
7. In Conclusion

Histogram Equalization

Image source: [Wikipedia](#)



In general histogram equalization consists in:

- Compute histogram of image
 - (May be of whole image, of just one channel, of every channel...)
- Compute cdf (addition scan) of histogram
- Compute this value for every pixel

$$s_k = \text{round} \left(\frac{CDF(r_k) \cdot (L - 1)}{n} \right)$$

(formula from [an article on medium](#))

- optional: as a preprocessing step, an rgb image may be converted to a color space like [hsl](#) for processing, then converted back to rgb after processing

what we're doing is

- Convert the image from RGB(A) to HSL(A) (parallel pattern: **map**)
- Compute histogram of L channel of HSL(A) image (parallel pattern: **histogram**)
- Compute CDF of L channel (parallel pattern: **scan**)
- Normalize L channel of image using histogram CDF (parallel pattern: **map**)
- Turn HSL(A) image back into RGB(A) (parallel pattern: **map**)

Implementation

- GPU version developed using Python and PyOpenCL
 - OpenCL for running locally (issues with remote development environments)

closest alternative to CUDA, mimics most of its API

- Python due to development timing constraints (C port was planned but could not be made in time)
- Slides will still use CUDA terms (thread, thread block, grid, ...) instead of the corresponding OpenCL terms (work item, work group, ndrange, ...)
- CPU version developed using NumPy
 - Fairer comparison with GPU Python code than doing it in C or C++

(any potential overhead from python runtime will be present in both the parallel and the sequential version)

PyOpenCL code laid out according to following pipeline

- host image is read from file, host histogram is initialized to all zeros
- host image and histogram are copied over to device
- device image is converted from rgb(a) to hsl(a) inplace
- histogram of L channel in device image computed into device histogram
- compute addition scan of device histogram inplace
- device image L channel normalized inplace

- **Color space conversion:**
 - every thread acts on one pixel of the image, changing it inplace
 - 1d grid, image is treated as a flat array of pixels as no 2d specific behaviour is required due to the pixelwise nature of the operation

- every thread acts on one pixel of the image
- thread blocks hold private partial histograms in local memory to avoid contention in global memory
- the first 256 threads in a block flush the partial sums to the global sum, after a barrier ensuring all elements have been counted in block local partial sum
- contention with multiple threads needing to increment values in the same array (histogram) handled through atomic operations
- 1d array, image treated as flat array of pixels, same reasoning as previous slide

```
__kernel void full_hist(const __global uchar* input,
                        __global uint* global_hist,
                        __local uint* local_hist,
                        const uint input_size,
                        const uint hist_size) {
    const uint gi = get_global_id(0);
    const uint li = get_local_id(0);
    if(li < hist_size) local_hist[li] = 0;
    barrier(CLK_GLOBAL_MEM_FENCE | CLK_LOCAL_MEM_FENCE);
    if(gi < input_size)
        atomic_add(&local_hist[input[gi]], 1);
    barrier(CLK_GLOBAL_MEM_FENCE | CLK_LOCAL_MEM_FENCE);
    if(li < hist_size)
        atomic_add(&global_hist[li], local_hist[li]);
}
```

- 2 level hierarchical scan, handles up to max_block_size^2 elements
- kogge-stone and brent-kung versions have been made
- brent-kung used for both levels, though given input size (256 elements) the choice was arbitrary and had no performance impact
- OpenCL has no mechanism to have grid level synchronization, hierarchical scan therefore implemented as 3 kernels
- (code in next slide for one level of ks since bk code didn't fit)

```
__kernel void ks_block_scan(__global uint* global_data,
                           const uint global_data_size,
                           __local  uint* local_data,
                           const uint local_data_size) {
    const uint global_idx = get_global_id(0);
    const uint local_idx  = get_local_id(0);
    const bool is_active  = (global_idx < global_data_size);
    local_data[local_idx] = is_active ? global_data[global_idx] : 0;
    barrier(CLK_LOCAL_MEM_FENCE);
    for(uint stride = 1; stride < local_data_size; stride *= 2) {
        const uint tmp = (local_idx >= stride) ? local_data[local_idx - stride] : 0;
        barrier(CLK_LOCAL_MEM_FENCE);
        if (local_idx >= stride) local_data[local_idx] += tmp;
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    if(is_active) global_data[global_idx] = local_data[local_idx];
}
```

- pinco pallino
- si ruppe il nasino
- ei era un cogliole
- ei era un cretino
- pinco pallino
- si spezzo il bacino
- lo presi a mazzate
- con un tavolino

Benchmarks

Conclusions

